

Unsupervised Learning

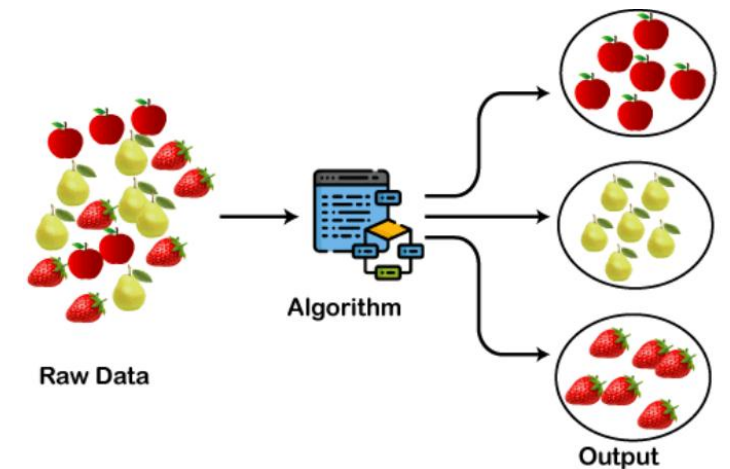
Week # 10

Course Instructor: Mahrukh Khan

Unsupervised Learning

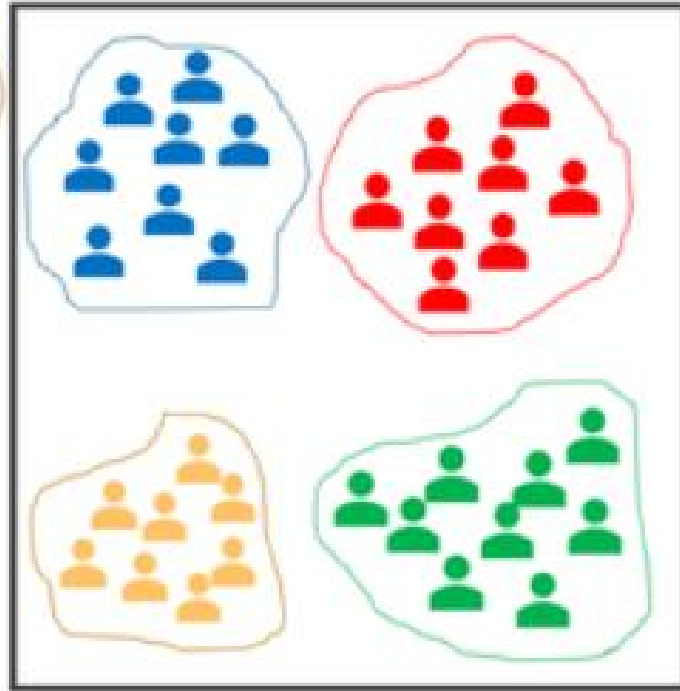
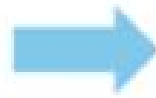
- Unsupervised learning is a type of machine learning that analyzes and models data without labelled responses or predefined categories.
- unsupervised learning algorithms work solely with input data and aim to discover hidden patterns, structures or relationships within the dataset independently, without any human intervention or prior knowledge of the data's meaning.

Supervised Learning	Unsupervised Learning
Has labels	No labels
Predicts known outcomes	Discovers hidden structure
Examples: spam detection, house price prediction	Examples: customer segmentation, topic discovery





Trying to determine the appropriate audience for the product



Using clustering algorithms on the customer base



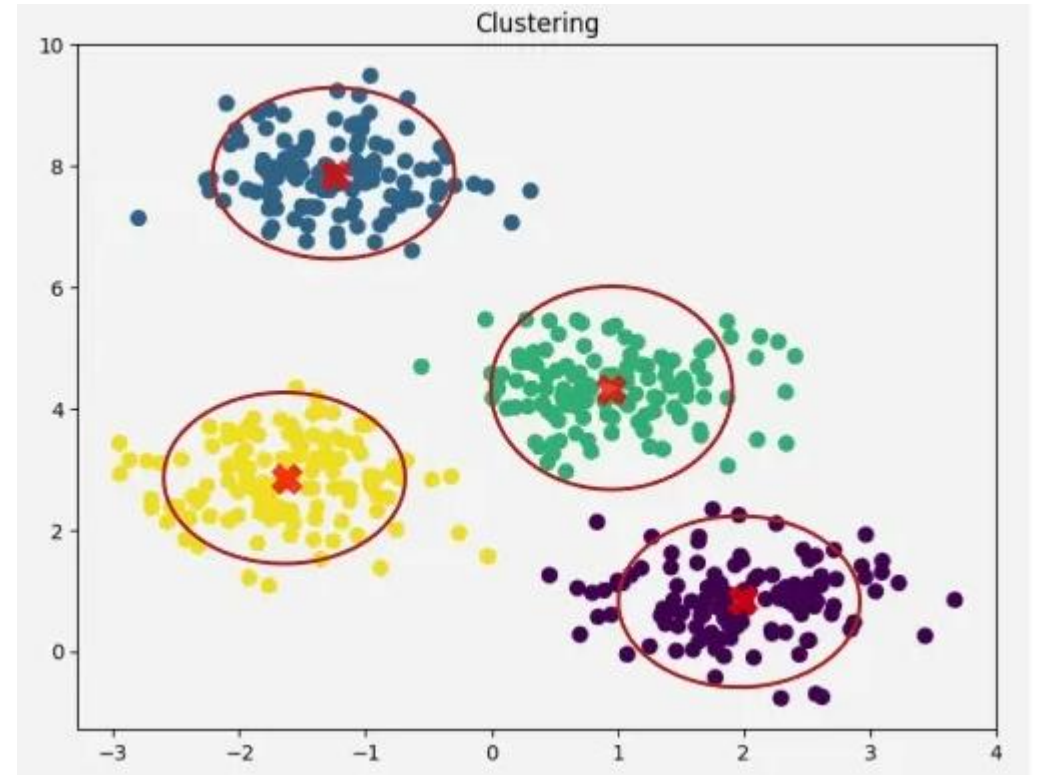
Selling the product to the targeted audience

Working of Unsupervised Learning

- **1. Collect Unlabelled Data**
 - Gather a dataset without predefined labels or categories.
- **2. Select an Algorithm**
 - Choose a suitable unsupervised algorithm such as clustering, association rule or dimensionality reduction based on the goal.
- **3. Train the Model on Raw Data**
 - Feed the entire unlabelled dataset to the algorithm. The algorithm looks for similarities, relationships or hidden structures within the data.
- **4. Group or Transform Data**
 - The algorithm organizes data into groups (clusters), rules or lower-dimensional forms without human input. Example: It may group similar animals together or extract key patterns from large datasets.
- **5. Interpret and Use Results**
 - Analyze the discovered groups, rules or features to gain insights or use them for further tasks like visualization, anomaly detection or as input for other models.

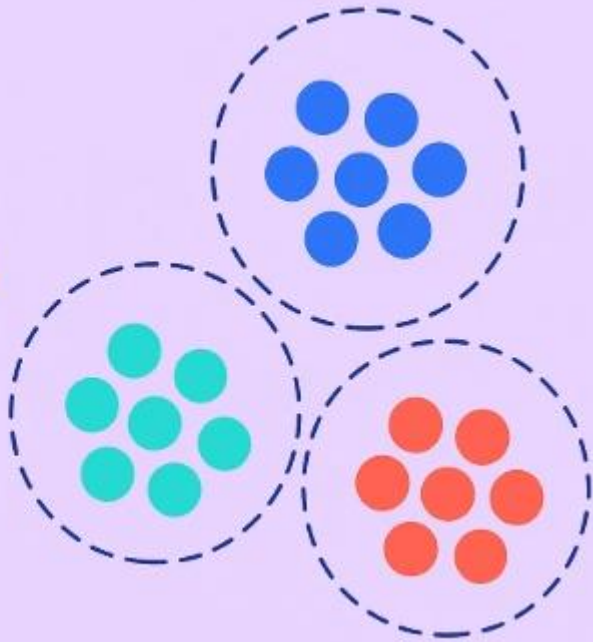
Unsupervised Learning Algorithms: Clustering

- **Goal:** Discover the natural grouping or structure in unlabelled` data without predefined categories.
- **How:** Data points are assigned to clusters based on similarity or distance measures.
- **Similarity Measures:** Can include Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.
- **Output:** Each group is assigned a cluster ID, representing shared characteristics within the cluster.



K means Clustering

What Is K-Means Clustering



- The k -means clustering algorithm is an unsupervised machine learning technique that partitions data into K distinct clusters based on similarity.
- Each cluster is defined by its center (mean), and the algorithm iteratively assigns each data point to the nearest cluster center, updating centers until the solution converges.

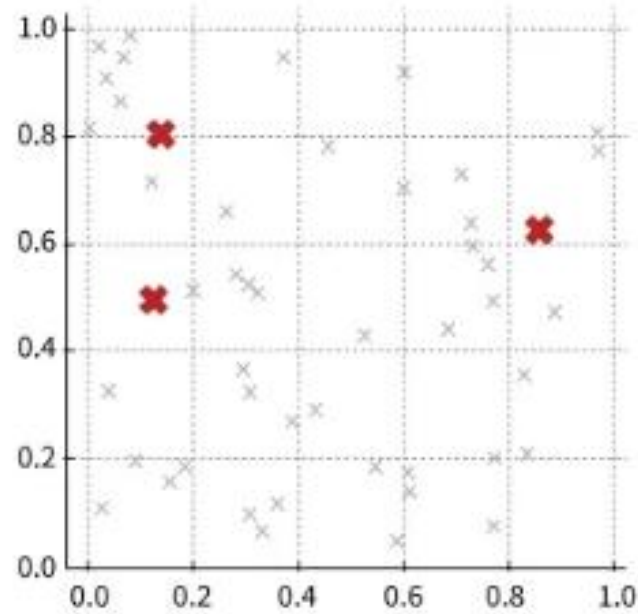
Working of K-Means Clustering

- Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups.
- " k " represents the number of groups or clusters we want to classify our items into.
- The algorithm will categorize the items into " k " groups or clusters of similarity.
- To calculate that similarity we will use the Euclidean distance as a measurement.
- The algorithm works as follows:
 - **Initialization:** We begin by randomly selecting k cluster centroids.
 - **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
 - **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
 - **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

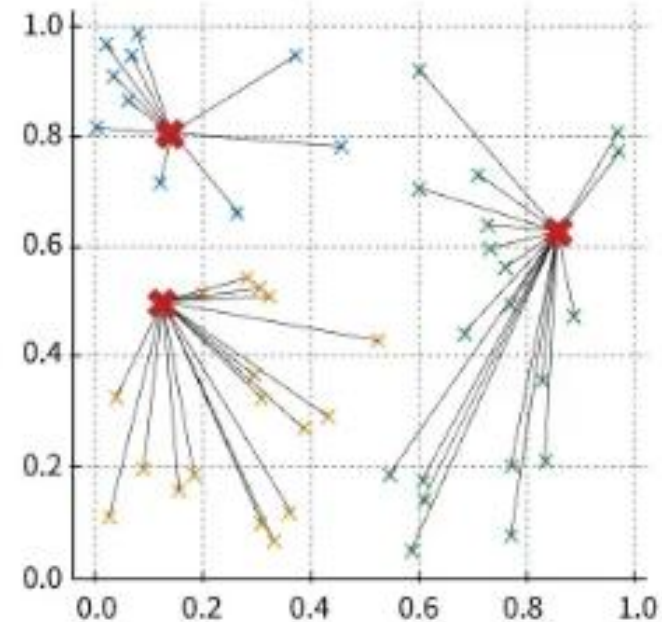
✖ Centroids ✕ Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

✖ Centroids ✕ Cluster 1
✕ Cluster 2 ✕ Cluster 3



Update Centroids

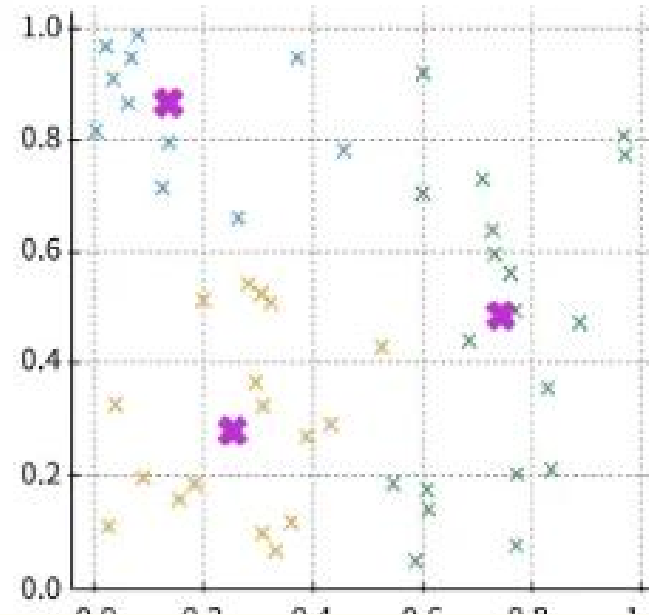
Centroids are recalculated as the mean of the points in each cluster

✱ New Centroids

✕ Cluster 1

✕ Cluster 2

✕ Cluster 3



Repeat Until Convergence

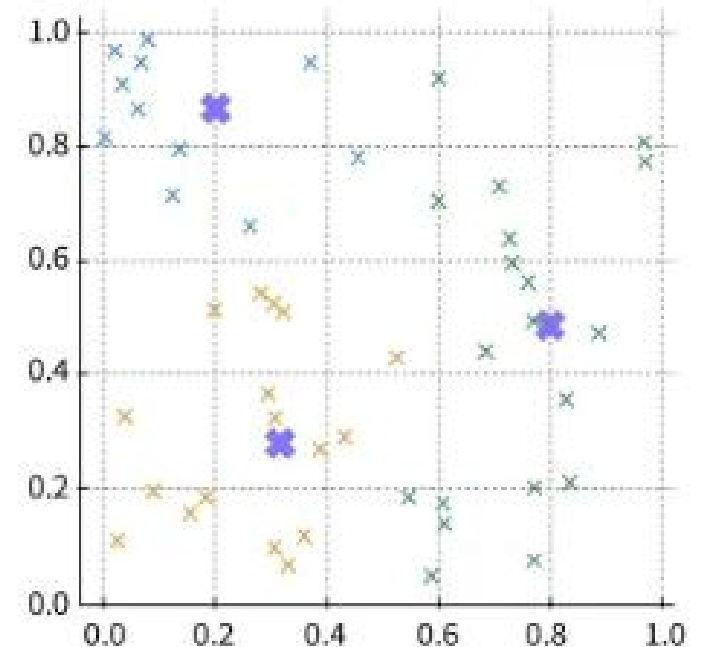
This process repeats until the centroids stabilize and do not move further.

✱ Final Centroids

✕ Cluster 1

✕ Cluster 2

✕ Cluster 3



Example

- Cluster 6 customers into **3 groups** based on their **Income** and **Spending Score**.
- **Step 1. Initialization**
 - Let's randomly select **3 initial centroids** from the data:
 - **Centroid₁ (C₁)** = A → (15, 1)
 - **Centroid₂ (C₂)** = D → (60, 8)
 - **Centroid₃ (C₃)** = F → (70, 10)

Customer	Income (\$000)	Spending Score
A	15	1
B	18	2
C	25	3
D	60	8
E	62	9
F	70	10

Example

- **Step 2. Compute Euclidean distances**

- $$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Customer	(Income, Spend)	dist→ C ₁ (15,1)	dist→ C ₂ (60,8)	dist→ C ₃ (70,10)	Nearest
A	(15,1)	0	45.5	55.7	C ₁
B	(18,2)	3.16	42.4	52.8	C ₁
C	(25,3)	10.2	35.4	45.5	C ₁
D	(60,8)	45.5	0	10.2	C ₂
E	(62,9)	47.7	2.24	8.25	C ₂
F	(70,10)	55.7	10.2	0	C ₃

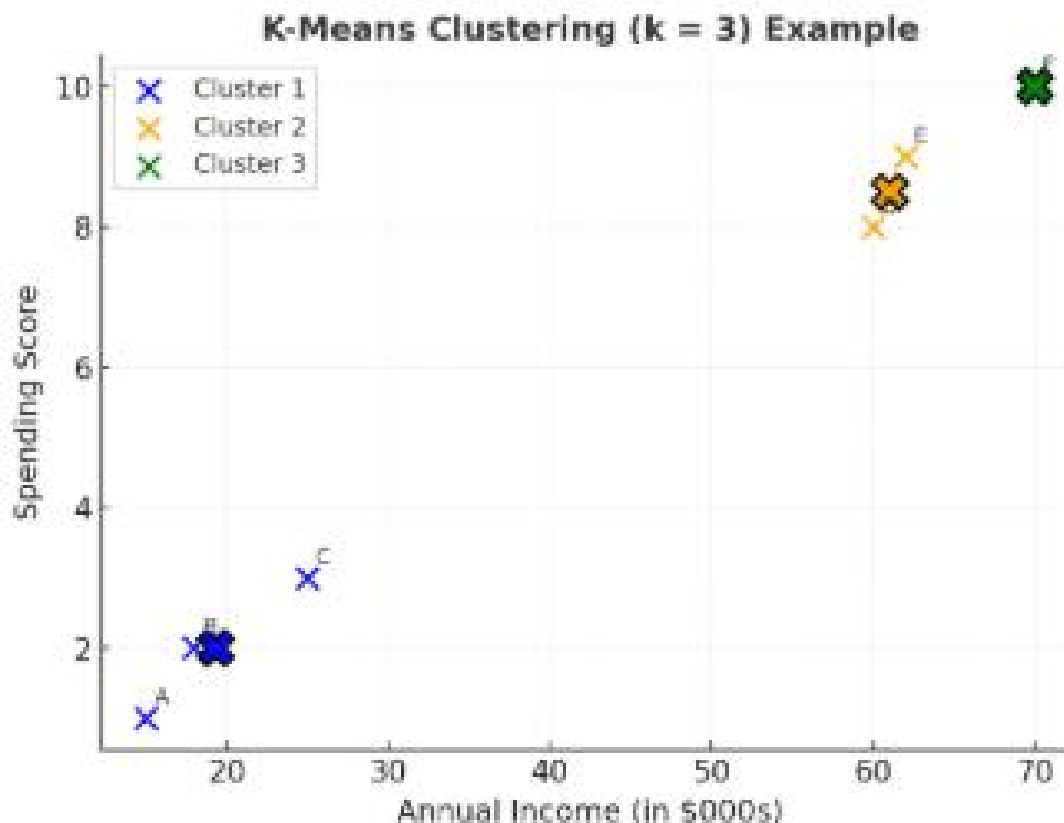
Cluster 1 (C₁): A, B, C

Cluster 2 (C₂): D, E

Cluster 3 (C₃): F

Example

- **Step 3. Update centroids**
 - Compute the mean of each cluster:
- **Cluster 1 (A,B,C):**
 - Mean Income = $(15 + 18 + 25)/3 = 19.3$
 - Mean Spending = $(1 + 2 + 3)/3 = 2.0$
→ New $C_1 = (19.3, 2.0)$
- **Cluster 2 (D,E):**
 - Mean Income = $(60 + 62)/2 = 61.0$
 - Mean Spending = $(8 + 9)/2 = 8.5$
→ New $C_2 = (61.0, 8.5)$
- **Cluster 3 (F):**
 - Only one point (70,10)
→ New $C_3 = (70,10)$



- Step 4. Reassign points (check again)

Customer	(Income, Spend)	dist→C ₁ (19.3,2)	dist→C ₂ (61,8.5)	dist→C ₃ (70,10)	Cluster
A	(15,1)	4.39	46.0	55.9	C ₁
B	(18,2)	1.30	43.0	53.8	C ₁
C	(25,3)	5.83	36.1	46.1	C ₁
D	(60,8)	44.0	1.12	10.2	C ₂
E	(62,9)	46.0	1.12	8.25	C ₂
F	(70,10)	54.2	9.06	0	C ₃

- No changes — **converged.**

Example:

- A retail store wants to segment customers based on their **annual income (\$000)** and **spending score (1–10)** to design marketing campaigns.
- Initial clusters: C1 c2, c3

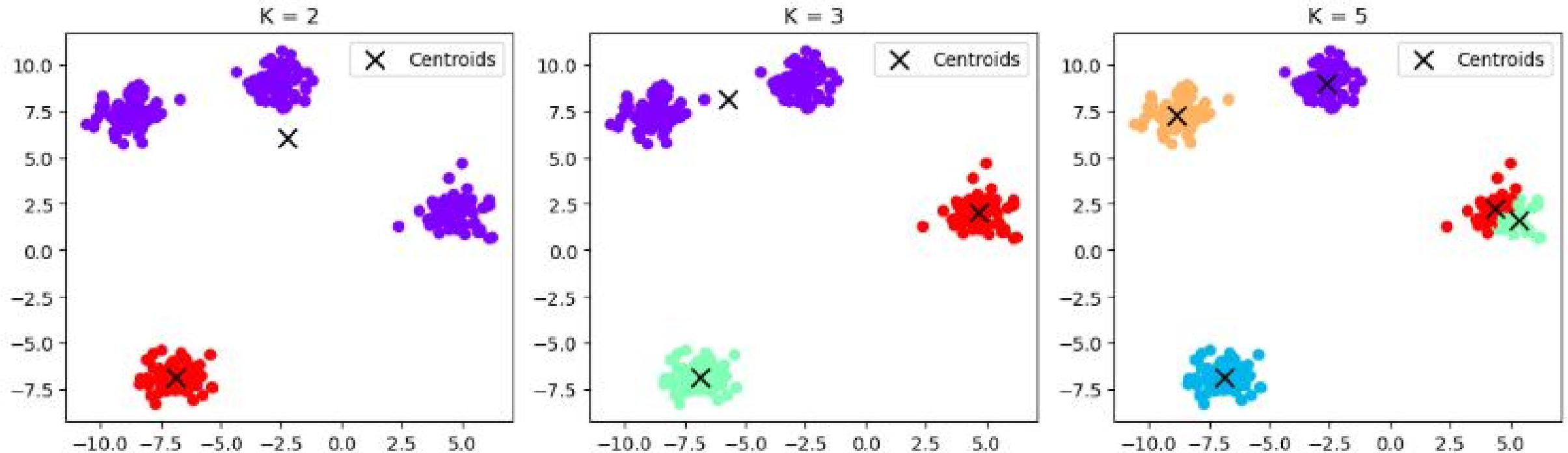
Customer	Income	Spending Score
A	15	2
B	18	3
C	22	1
D	30	4
E	35	8
F	55	9
G	58	10
H	60	7
I	62	5

- C_1 A, B, C(18.3, 2.0)Low income, low spending customers
 - C_2 D, E(32.5, 6.0)Mid income, active spenders
 - C_3 F, G, H, I(58.8, 7.8)High income, high spending customers
-
- <https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>
 - <https://www.linkedin.com/pulse/implementing-k-means-algorithm-python-alex-o-neill/>

Challenges with K-Means Clustering

- **Choosing the Right Number of Clusters (k):** One of the biggest challenges is deciding how many clusters to use.
- **Sensitive to Initial Centroids:** The final clusters can vary depending on the initial random placement of centroids.
- **Non-Spherical Clusters:** K-Means assumes that the clusters are spherical and equally sized. This can be a problem when the actual clusters in the data are of different shapes or densities.
- **Outliers:** K-Means is sensitive to outliers, which can distort the centroid and, ultimately, the clusters.

- “If I don’t know how many clusters exist in my data, how can I find K?”

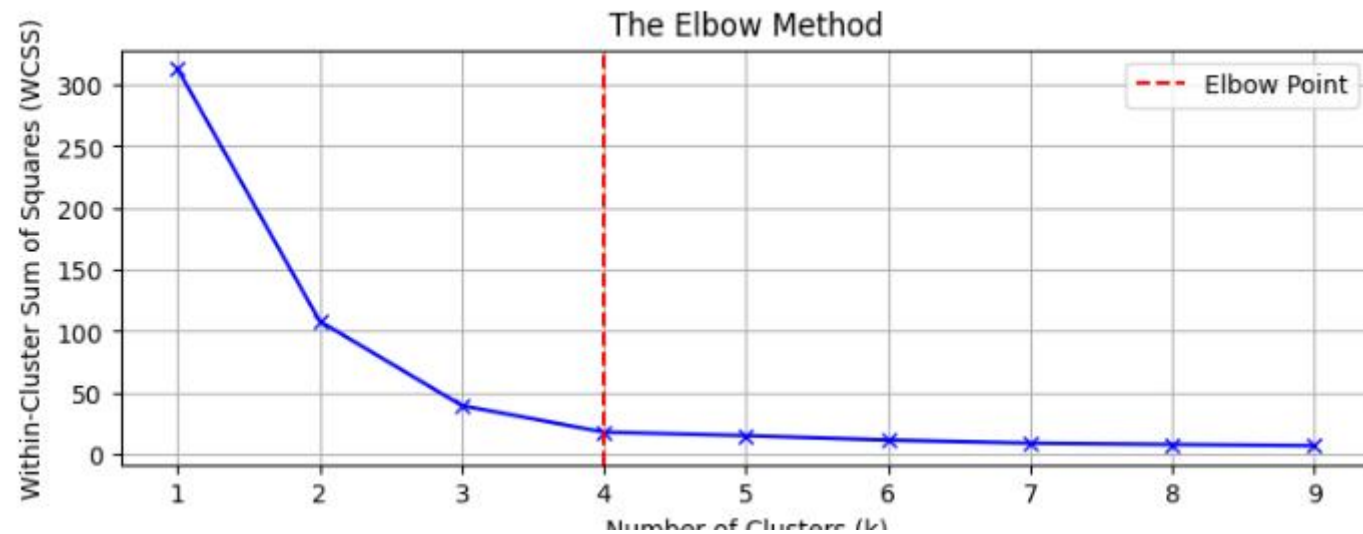


Elbow Method for optimal value of k in KMeans

- Within-Cluster Sum of Squares (WCSS)
- For each K, K-Means groups the data into K clusters.
- We calculate **WCSS** = how tightly points are grouped inside clusters.
- As K increases, WCSS decreases (clusters get smaller and tighter). But after a point, the gain becomes small — that's the *elbow*.

$$WCSS = \sum_{i=1}^k \sum_{j=1}^{n_i} \text{distance}(x_j^{(i)}, c_i)^2$$

- Where , $\text{distance}(x_j^{(i)}, c_i)$ represents the distance between the j^{th} data point $x_j^{(i)}$ in cluster i and the centroid c_i of that cluster.



- As we increase k , the WCSS typically decreases because we're creating more clusters, which tend to capture more data variations. However, there comes a point where adding more clusters results in only a marginal decrease in WCSS. This is where we observe an "elbow" shape in the graph.
- Elbow = tradeoff between simplicity (fewer clusters) and accuracy (lower WCSS).

Cluster Evaluation Metrics

- Quality clusters contain at least two properties:
 - **All data points within a cluster should be similar.**
 - **Clusters should be distinct from each other.**

“When we form clusters, how do we know if they’re *good* or *bad*?”

Distortion

- Measures how *far* points are from their cluster centers.

$$\text{Distortion} = \frac{1}{N} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- “average within-cluster spread.”
- Lower distortion → tighter clusters → better clustering.

Inertia

- **Inertia** is the **sum of squared distances** (not averaged).

$$\text{Inertia} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- “total distortion” for all clusters combined.

Example

- Compute Final Inertia (WCSS)

- **Cluster 1**

- A: $(15,2) \rightarrow (18.33,2) \Rightarrow (-3.33)^2 + 0^2 = 11.09$
B: $(18,3) \rightarrow \text{diff } (-0.33, 1) \Rightarrow 0.11 + 1 = 1.11$
C: $(22,1) \rightarrow \text{diff } (3.67, -1) \Rightarrow 13.46 + 1 = 14.46$
→ **Total = 26.66**

- **Cluster 2**

- D: $(30,4) \rightarrow \text{diff } (-2.5, -2) \Rightarrow 6.25 + 4 = 10.25$
E: $(35,8) \rightarrow \text{diff } (2.5, 2) \Rightarrow 6.25 + 4 = 10.25$
→ **Total = 20.5**

Cluster	Centroid (Income, Spending)	Members
C ₁	(18.33, 2.00)	A, B, C
C ₂	(32.5, 6.00)	D, E
C ₃	(58.75, 7.75)	F, G, H, I

- **Cluster 3**

- F: $(55,9) \rightarrow \text{diff } (-3.75, 1.25) \Rightarrow 14.06 + 1.56 = 15.62$
G: $(58,10) \rightarrow \text{diff } (-0.75, 2.25) \Rightarrow 0.56 + 5.06 = 5.62$
H: $(60,7) \rightarrow \text{diff } (1.25, -0.75) \Rightarrow 1.56 + 0.56 = 2.12$
I: $(62,5) \rightarrow \text{diff } (3.25, -2.75) \Rightarrow 10.56 + 7.56 = 18.12$
→ **Total = 41.42**

Overall Inertia = 26.66 + 20.5 + 41.42 = 88.58

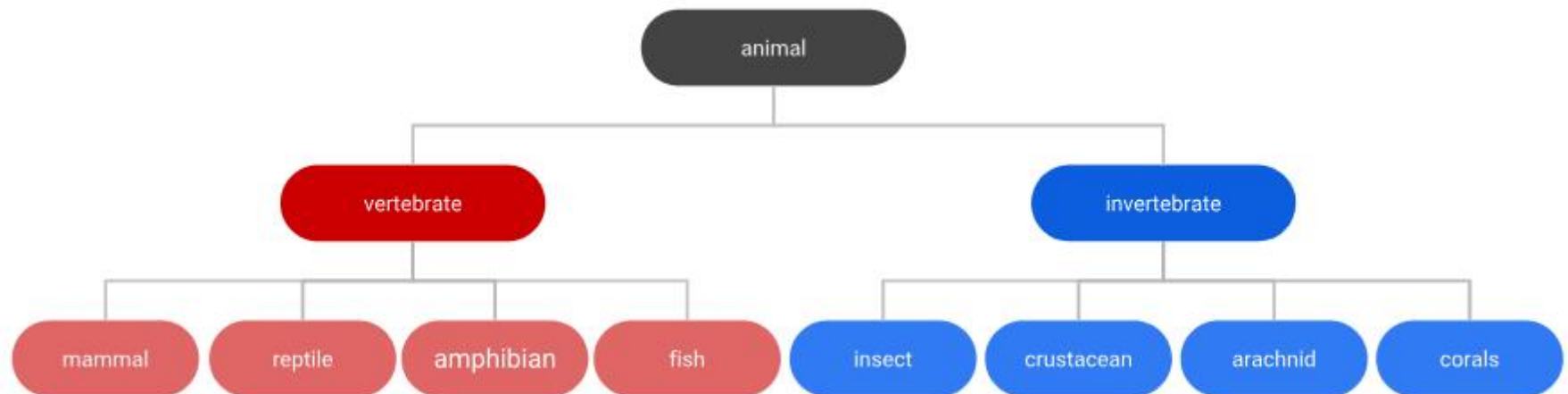
Distortion = 88.58 / 9 = 9.84

Types of Clustering

- Centroid based (k-means)
- Density-based clustering
- Distribution-based clustering
- Hierarchical clustering

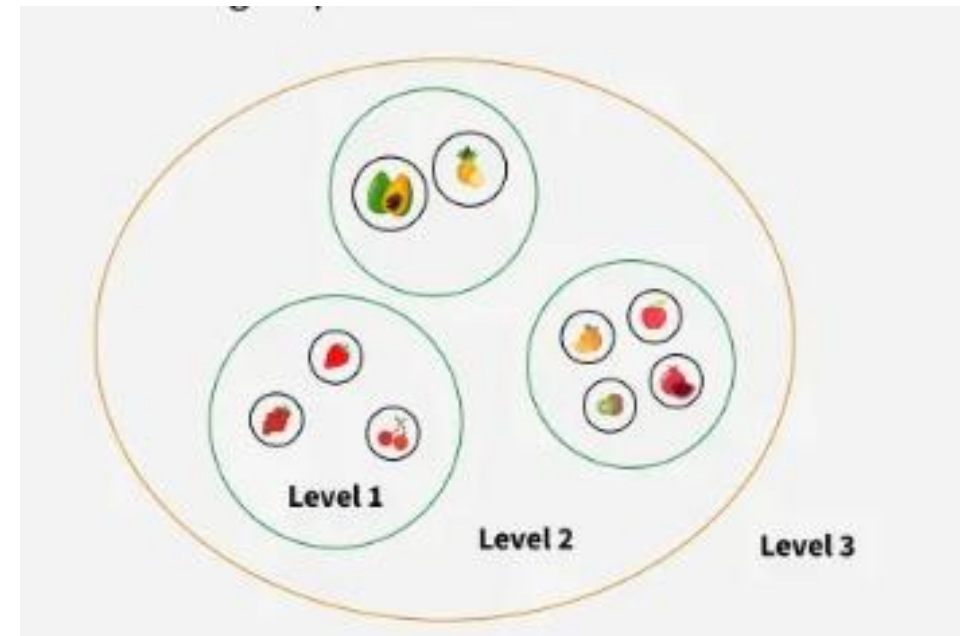
Hierarchical Clustering

- Many complex physical, biological and social systems have nested hierarchical structures.
- Hierarchical clustering algorithms use the concept of a dissimilarity matrix to decide which clusters to merge or divide.
- Dissimilarity is the distance between two data points as measured by a chosen linkage method. The values in a dissimilarity matrix express:
 - The distance, a Euclidean distance as an example, between single points in a set
 - A linkage clustering criterion, which specifies dissimilarity as a function of the pairwise distances of points across sets



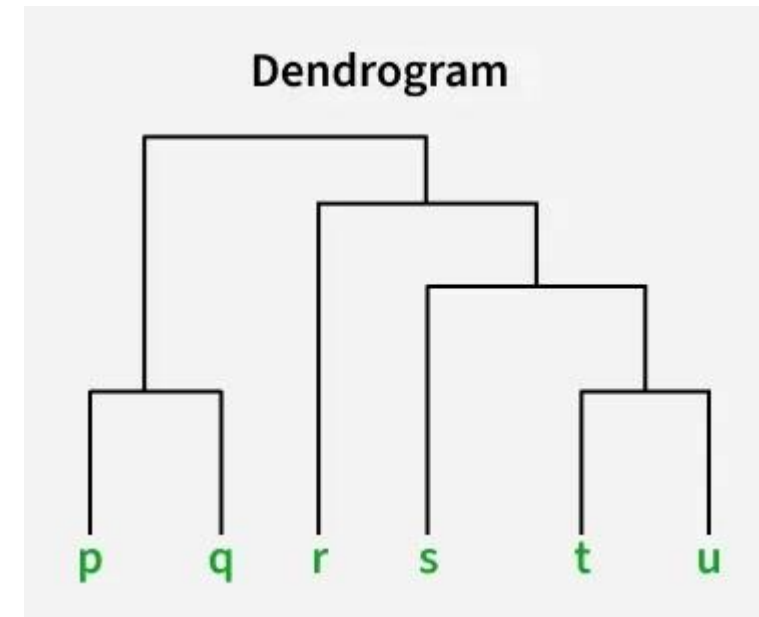
Hierarchical Clustering

- Imagine we have four fruits with different weights:
 - an apple (100g), a banana (120g), a cherry (50g) and a grape (30g).
- Hierarchical clustering starts by treating each fruit as its own group.
 - Start with each fruit as its own cluster.
 - Merge the closest items: grape (30g) and cherry (50g) are grouped first.
 - Next, apple (100g) and banana (120g) are grouped.
 - Finally, these two clusters merge into one.



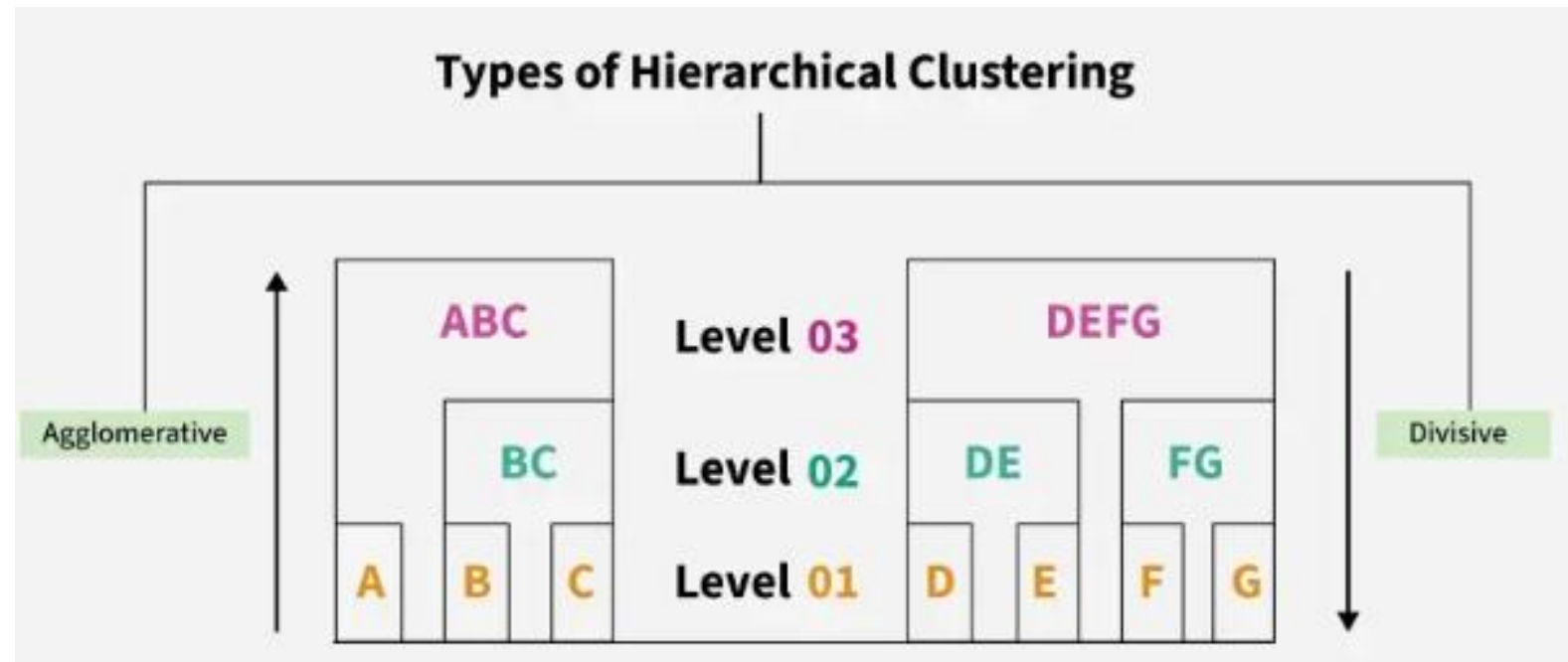
Dendrogram

- A dendrogram is like a family tree for clusters. It shows how individual data points or groups of data merge together. The bottom shows each data point as its own group and as we move up, similar groups are combined. The lower the merge point, the more similar the groups are. It helps us see how things are grouped step by step.



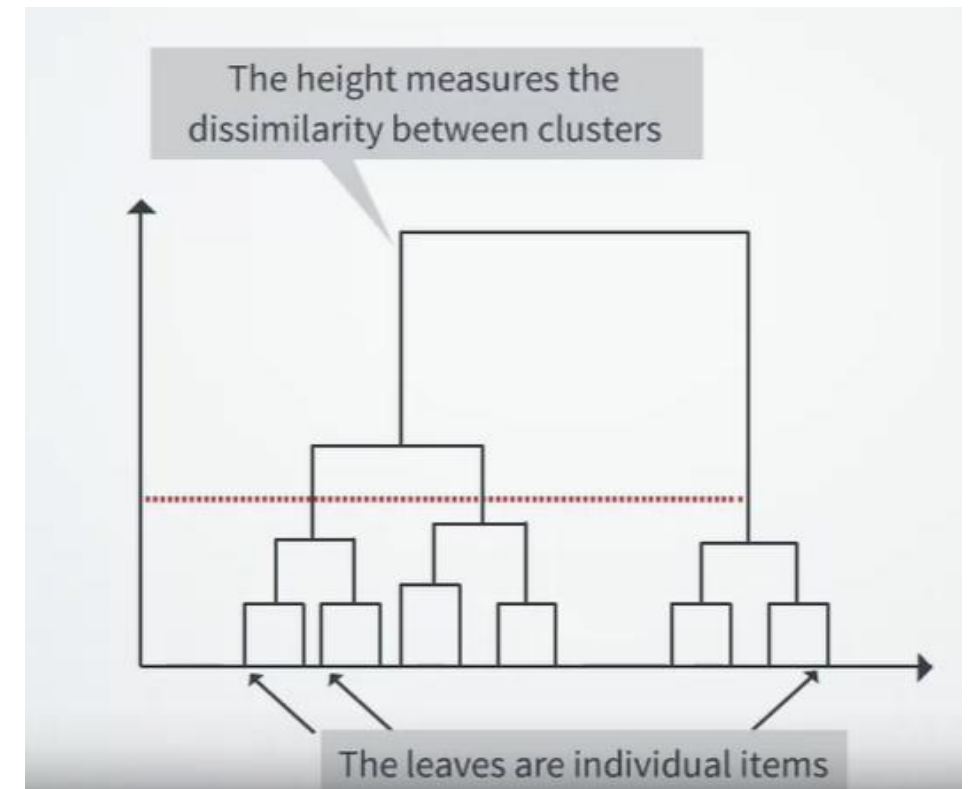
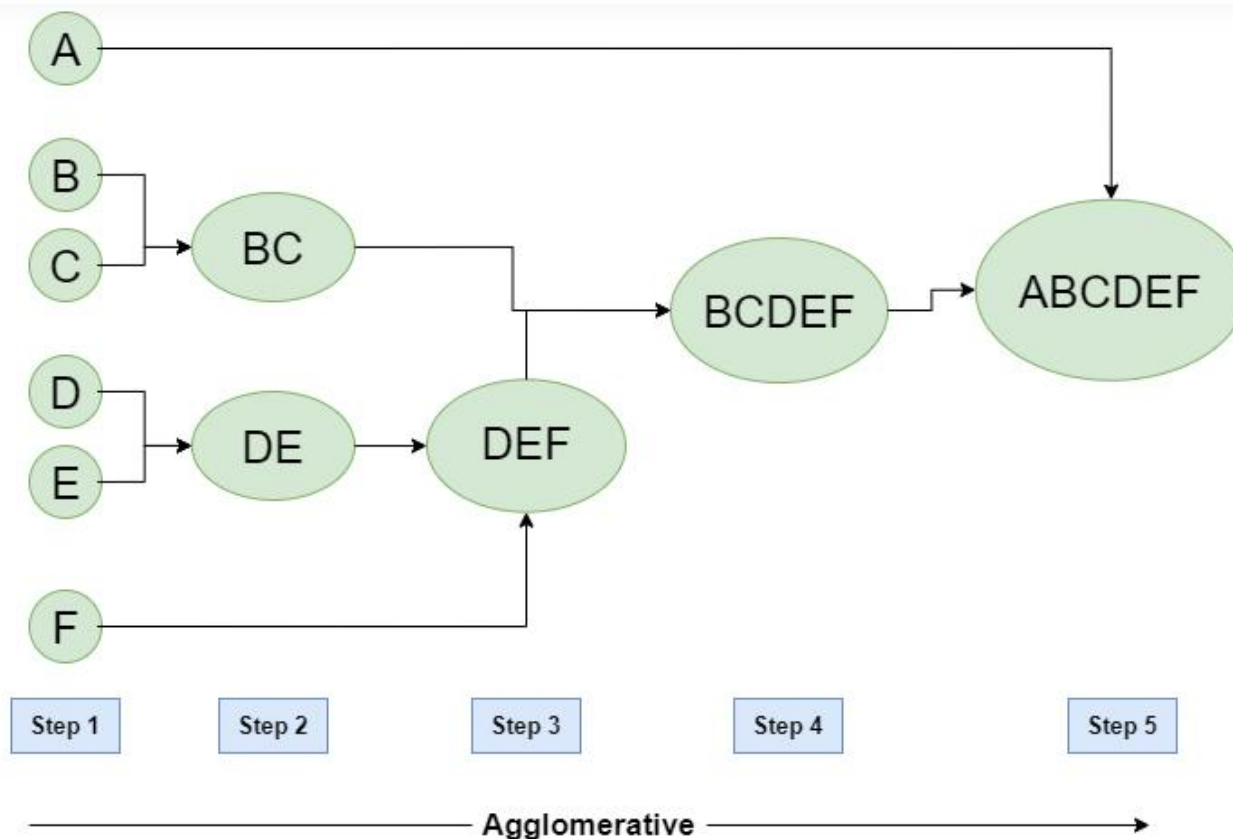
Types of Hierarchical Clustering

- [Agglomerative](#) (Bottom-up): Start with each point as a cluster; iteratively merge closest clusters.
- [Divisive](#) (Top-down): Start with one cluster; iteratively split into smaller clusters.



Hierarchical Agglomerative Clustering

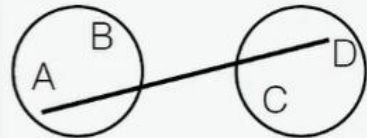
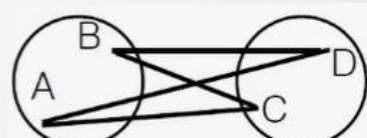
- Start with **each data point as its own cluster**, then **merge the most similar clusters step by step** until only one cluster (or desired number) remains.



Working and Implementation of Hierarchical Clustering

1. **Start with individual points:** Each data point is its own cluster. For example, if we have 5 data points we start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
6. **Create a dendrogram:** As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.

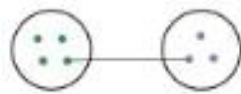
Linkage Methods:(How to Measure Cluster Distance)

Name	Definition	
Single linkage clustering	$\min \{d(a,b) : a \in A, b \in B\}$	
Complete linkage clustering	$\max \{d(a,b) : a \in A, b \in B\}$	
Average linkage clustering	$\frac{1}{ A B } \sum_{a \in A} \sum_{b \in B} d(a,b)$	

Note that each linkage method generates different clusters from the same dataset. Selecting the appropriate linkage clustering method depends on factors such as the type of data being clustered, data density, cluster shape and whether there are outliers or noise in the dataset.

Distance Metrics

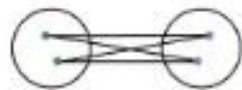
- While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity. But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:



Minimum Distance



Maximum Distance



Group Average



Ward's Method

- **Min Distance:** Find the minimum distance between any two points of the cluster.
- **Max Distance:** Find the maximum distance between any two points of the cluster.
- **Group Average:** Find the average distance between every two points of the clusters.
- **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.



Example:

Point	X	Y
A	1	1
B	2	1
C	5	4
D	6	5
E	8	4

	A	B	C	D	E
A	0	1.0	5.0	6.4	7.6
B	1.0	0	4.2	5.7	7.0
C	5.0	4.2	0	1.4	3.0
D	6.4	5.7	1.4	0	2.2
E	7.6	7.0	3.0	2.2	0

- Step 1: Compute pairwise distances,

Agglomerative Single Link Algorithm

- Single-nearest distance or single linkage is the agglomerative method that uses the distance between the closest members of the two clusters.
- Smallest distance = **1.0** between **A and B**
→ Merge **A and B** → Cluster₁ = (A,B)
- Now compute distance between new cluster (A,B) and other points:
Use **minimum** distance rule.

From	To	Distance (min of members)
(A,B)	C	$\min(5.0, 4.2) = 4.2$
(A,B)	D	$\min(6.4, 5.7) = 5.7$
(A,B)	E	$\min(7.6, 7.0) = 7.0$

Find the next smallest distance
 ✓Smallest = **1.4** between **C and D**
 Merge → **(C D)**

	(A B)	C	D	E
(A B)	0	4.2	5.7	7.0
C	4.2	0	1.4	3.0
D	5.7	1.4	0	2.2
E	7.0	3.0	2.2	0

Update distances again

To	From (C D)	min distance
(A B)	min(A–C = 5.0, A–D = 6.4, B–C = 4.2, B–D = 5.7)	4.2
E	min(C–E = 3.0, D–E = 2.2)	2.2

	(A B)	(C D)	E
(A B)	0	4.2	7.0
(C D)	4.2	0	2.2
E	7.0	2.2	0

Find the next smallest distance
 ✓Smallest = **2.2** between **C,D and E**
 Merge → **(C, D,E)**
 Merge at **distance = 4.2**
Final Cluster (A, B, C, D, E)

To	From	min distance
(A B) – (C D E)	min(A–C = 5.0, A–D = 6.4, A–E = 7.6, B–C = 4.2, B–D = 5.7, B–E = 7.0)	4.2

Agglomerative Complete Link Algorithm

- Single-nearest distance or single linkage is the agglomerative method that uses the distance between the closest members of the two clusters.
- Complete link = the **maximum** distance between any pair of points in the two clusters.

Find the next smallest distance

✓Smallest = 1.4 between C and D

Merge → (C D)

	(A B)	C	D	E
(A B)	0	5	6.4	7.6
C	5	0	1.4	3.0
D	6.4	1.4	0	2.2
E	7.6	3.0	2.2	0

Update distances again

To	From (C D)	min distance
(A B)	max(A–C = 5.0, A–D = 6.4, B–C = 4.2, B–D = 5.7)	6.4
E	max(C–E = 3.0, D–E = 2.2)	3.0

	(A B)	(C D)	E
(A B)	0	6.4	7.0
(C D)	6.4	0	3.0
E	7.0	3.0	0

Find the next smallest distance

✓Smallest =3 between C,D and E

Merge → (C, D,E)

Merge at distance = 7.6

Final Cluster (A, B, C, D, E)

To	From	min distance
(A B) – (C D E)	min(A–C = 5.0, A–D = 6.4, A–E = 7.6, B–C = 4.2, B–D = 5.7, B–E = 7.0)	7.6

Agglomerative Average Link Algorithm

- Average-average distance or average linkage is the method that involves looking at the distances between all pairs and averages all of these distances. This is also called **Universal Pair Group Mean Averaging**.

To	Formula
C	$(A-C + B-C)/2 = (5.0 + 4.2)/2 = 4.6$
D	$(A-D + B-D)/2 = (6.4 + 5.7)/2 = 6.05$
E	$(A-E + B-E)/2 = (7.6 + 7.0)/2 = 7.3$

Find the next smallest distance
 ✓Smallest = **1.4** between **C and D**
 Merge → **(C D)**

	(A B)	C	D	E
(A B)	0	4.6	6.05	7.3
C	4.6	0	1.4	3.0
D	6.05	1.4	0	2.2
E	7.3	3.0	2.2	0

Update distances again

To	From (C D)	min distance
(A B)	Average of all 2×2 = (A–C, A–D, B–C, B–D) = (5.0 + 6.4 + 4.2 + 5.7)/4 = 5.33	5.33
E	(C–E + D–E)/2 = (3.0 + 2.2)/2 = 2.6	2.6

	(A B)	(C D)	E
(A B)	0	5.33	7.3
(C D)	5.33	0	2.6
E	7.3	2.6	0

Find the next smallest distance
 ✓Smallest =2.6 between **C,D and E**
 Merge → **(C, D,E)**

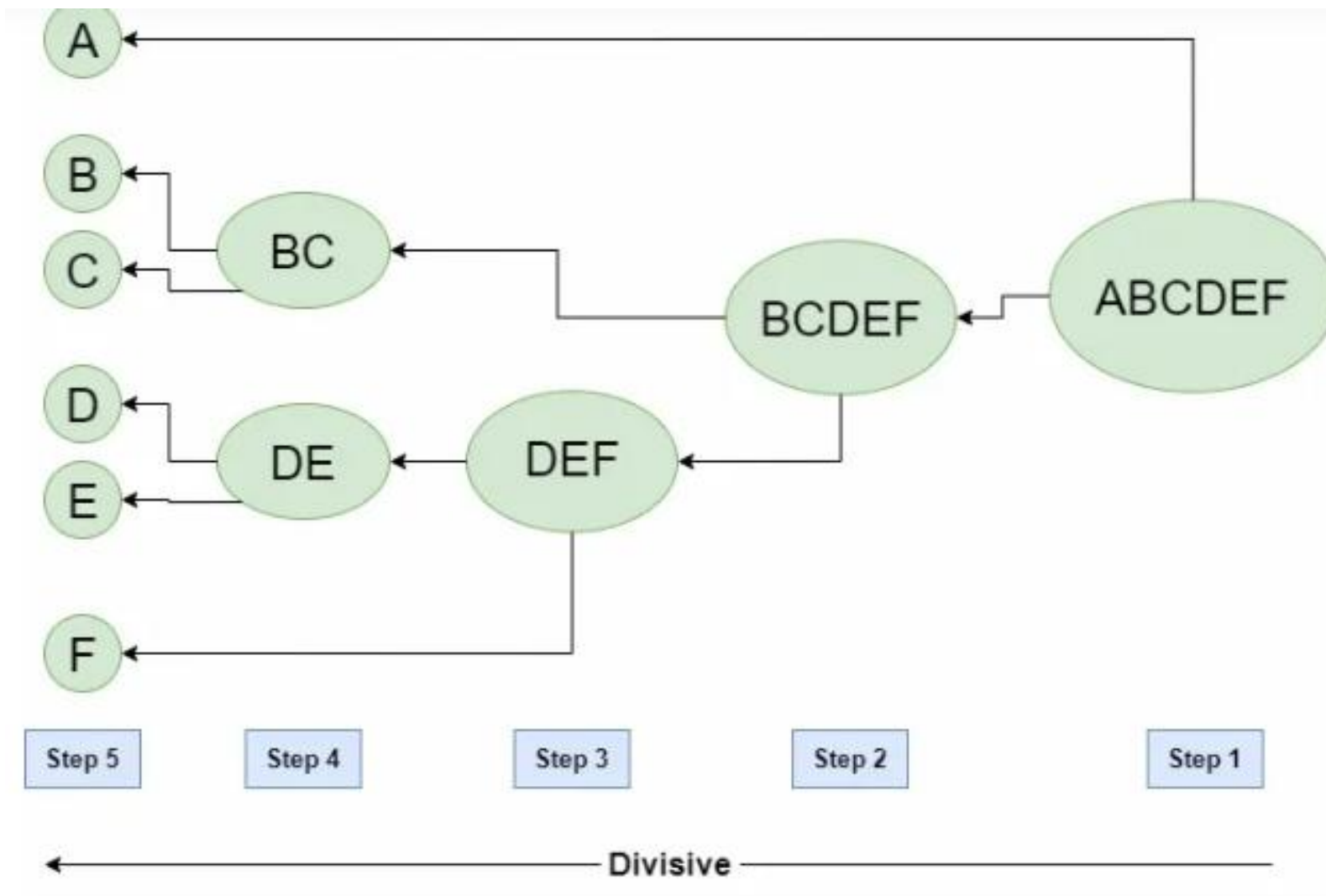
Merge at **distance = 5.98**
Final Cluster (A, B, C, D, E)

To	From	min distance
(A B) – (C D E)	avg(A–C = 5.0, A–D = 6.4, A–E = 7.6, B–C = 4.2, B–D = 5.7, B–E = 7.0)	35.9/6=5.98

Hierarchical Divisive clustering

- Divisive clustering is also known as a top-down approach.
- Divisive = start with **all points in one cluster**, and **split** recursively into smaller clusters.
- Step 1 – Split the cluster with the largest **distance spread**
- We look for the **two most dissimilar points** to start the split.
- The **maximum distance** is **A–E = 7.6**. So we split based on that dissimilarity.

	A	B	C	D	E
A	0	1.0	5.0	6.4	7.6
B	1.0	0	4.2	5.7	7.0
C	5.0	4.2	0	1.4	3.0
D	6.4	5.7	1.4	0	2.2
E	7.6	7.0	3.0	2.2	0



Workflow for Hierarchical Divisive clustering :

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.

Hierarchical Divisive clustering

- Option 1 — Split into {A} and {B, C, D, E}
 - Let's see if {B, C, D, E} is cohesive:
 - Average distance in {B,C,D,E} $\approx$ **3.9**
- Option 2 — Split into {A,B} and {C, D, E}
- A–B = 1.0 ✓ (very close)
- Within {C,D,E}: 1.4, 2.2, 3.0 $\rightarrow$ average = 2.2 ✓ (moderately close)
- Divisive clustering tries to make *both resulting clusters as compact as possible*. it minimizes within-cluster distance **and** reflects real structure in the data.

Pair	Distance
B–C	4.2
B–D	5.7
B–E	7.0
C–D	1.4
C–E	3.0
D–E	2.2

- Step 2 – Check which cluster is more heterogeneous (less cohesive)
- For **{A, B}** → distance = 1.0 → cohesive
- For **{C, D, E}** →
 - C–D = 1.4, C–E = 3.0, D–E = 2.2
 - Average = $(1.4 + 3.0 + 2.2)/3 = \mathbf{2.2}$ → less cohesive
 - So we **split {C, D, E}** next.
- Look for the **most distant pair** inside it:
- C–E = 3.0 → largest
So we split into two subclusters:
- (A B), (C D), (E)
- {A, B} → distance = 1.0 ✓ tight
- {C, D} → distance = 1.4 ✓ tight
- {E} → single point ✓ done

Term	Meaning	What it tells you
Cohesion	How close the points in a cluster are	Lower average distance = better cluster
High Cohesion	Points are tightly packed	Good cluster ✓
Low Cohesion	Points are spread out	Bad cluster ✗

- Biology / Bioinformatics->In DNA microarray data, genes that behave similarly under experiments cluster together
- Geography / Urban Planning->Used to group regions or cities based on multiple indicators: income, population, education, pollution levels, etc.
- Algorithmic Trading-> helping traders build portfolios, manage risk, and design trading strategies.
- Portfolio Construction and Optimization->how much money to invest in each asset (stock, bond, crypto, etc.)so that the overall portfolio gives high return with low risk.

